

预习报告

实验题目：简单函数绘图语言语法分析器的设计与实现

一、实验目的

1. 理解语法分析在编译系统中的作用，明确其输入为词法分析器输出的 **Token** 序列，输出为语法正确性判断或语法树结构。
2. 掌握上下文无关文法（CFG）的设计方法，能够针对函数定义与绘图指令建立无二义性的语法规则。
3. 掌握递归下降分析法的基本思想，建立“非终结符 - 递归函数”一一对应关系。
4. 学会在语法分析阶段处理运算符优先级与结合性，保证表达式分析结果正确。
5. 具备基础语法错误检测能力，能够对缺少分号、括号不匹配、关键字误用等错误给出清晰提示。

二、实验原理

语法分析是编译过程第二阶段。词法分析将字符流转为 **Token** 序列，语法分析根据 **CFG** 判断 **Token** 序列是否满足语言语法规则，并在需要时构建抽象语法树（AST）。

本实验采用自顶向下的递归下降方法，核心原理如下：

1. **文法分层**：将表达式文法按照优先级分层设计为 **Expr**（加减）、**Term**（乘除）、**Factor**（幂）、**Atom**（原子项），避免歧义并消除左递归。
2. **函数映射**：每个非终结符对应一个解析函数，例如 **Program** 对应 `parse_program()`，**Expr** 对应 `parse_expr()`。
3. **单 Token 前瞻**：使用 `current_token` 保存当前输入符号，通过 `advance()` 向前读取下一个 **Token**。
4. **匹配机制**：通过 `eat(expected_type, expected_value)` 校验当前 **Token** 是否符合期望；不符合时立即抛出语法错误。
5. **错误定位**：在错误信息中记录当前位置、期望符号与实际符号，便于调试。

三、语法规则与上下文无关文法设计

根据实验任务，目标语言语法如下：

- 程序由若干语句组成，每条语句后以分号结束。

- 语句支持两类：函数定义语句和绘图语句。
- 函数定义格式：func f(x) = x² + 1
- 绘图语句格式：plot f(x) range [0, 10]
- 表达式支持 + - * / ^ 和括号。

对应 CFG（无左递归）如下：

- Program → StmtList EOF
- StmtList → Stmt ; StmtList | epsilon
- Stmt → FuncDef | PlotStmt
- FuncDef → KEYWORD(func) IDENTIFIER LPAREN IDENTIFIER RPAREN OPERATOR(=) Expr
- PlotStmt → KEYWORD(plot) IDENTIFIER LPAREN IDENTIFIER RPAREN KEYWORD(range) LBRACKET Number COMMA Number RBRACKET
- Expr → Term Expr'
- Expr' → + Term Expr' | - Term Expr' | epsilon
- Term → Factor Term'
- Term' → * Factor Term' | / Factor Term' | epsilon
- Factor → Atom ^ Factor | Atom
- Atom → IDENTIFIER | Number | LPAREN Expr RPAREN
- Number → INTEGER | FLOAT

说明：

1. Expr' 和 Term' 使加减、乘除具备左结合特性。
2. Factor → Atom ^ Factor 使幂运算具备右结合特性。
3. StmtList 使用递归形式表达“语句序列”。

四、算法设计（预习）

1. 核心数据结构

- Token(type, value, pos): 记录类型、词素和值位置。
- ASTNode(kind, value, children): 统一表示语法树结点。

2. 语法分析器接口

- Parser(lexer): 接收词法分析器实例。
- parse(): 入口函数，调用 parse_program()。

3. 关键函数设计

- parse_program(): 循环读取语句直到 EOF。
- parse_stmt(): 根据关键字分派到函数定义或绘图指令。
- parse_expr()/parse_term()/parse_factor()/parse_atom(): 按优先级解析表达式。
- eat(): 进行严格匹配，失败立即报错。

4. 错误处理策略

- 缺少分号：在 `parse_stmt_list()` 中检测并提示“期望 ';'”。
- 括号不匹配：在 `parse_atom()` 中检测 (后是否存在对应)。
- 关键字误用：在 `parse_stmt()` 中检测非法语句起始符。

5. 语法树输出方案

- 使用层次缩进文本输出 AST，便于在终端中观察。
- 示例：Program -> FuncDef -> Expr -> Term ...

五、预期测试方案

计划使用至少 5 组测试数据：

1. 合法：单个函数定义。
2. 合法：函数定义 + 绘图语句组合程序。
3. 合法：包含括号与幂运算的复杂表达式。
4. 非法：缺少分号。
5. 非法：绘图语句缺少右方括号或关键字拼写错误。

预期结果：

- 合法输入能够完整输出语法树。
- 非法输入能够抛出带位置信息的 `SyntaxError`，并指明“期望/实际”符号。

六、预习总结

通过本次预习，已完成实验二语法设计与解析框架规划，明确了表达式优先级处理方法和错误检测点。后续实现阶段将以该 CFG 为依据完成 Parser 编码、词法衔接与系统测试，确保实验结果满足正确性和鲁棒性要求。